

The Big Picture

Before Learning Closures, You Must Know JavaScript Functions

01

Functions

02

Callback Functions

03

Higher Order Functions

04

Lexical Scope

05

Closures

Learning Flow

First understand functions

Then callback functions

Then higher order functions

Then lexical scope

Finally closures



Rule: Without understanding lexical scope, closures are impossible to understand.

What is a Callback Function?

Definition

A callback function is simply a function that is passed as an argument to another function.

Real Life Example

Imagine ordering food.

Customer places order

↓

Restaurant cooks food

↓

Delivery person calls you

↓

You receive food

The phone call happens after the food is ready.

That phone call is similar to a callback.

JavaScript Example

```
function greet(name) { console.log("Hello " + name); }  
  
function processUser(callback) { callback("Harinath"); }  
  
processUser(greet);
```

Output

Hello Harinath

Explanation

processUser()

↓

Receives greet()

↓

Calls greet()

↓

Prints Hello Harinath

What is a Higher Order Function?

Definition

A Higher Order Function is a function that

- accepts another function as an argument
- OR returns another function.

Example 1

```
function greet(name){ console.log("Hello " + name); }  
  
function process(callback){ callback("John"); }  
  
process(greet);
```

Here

`process()`

accepts another function.

Therefore

`process()`

is a Higher Order Function.

Example 2

```
function outer(){  
  
  return function(){ console.log("Welcome"); }  
  
}  
  
const result = outer();  
  
result();
```

Output

Welcome

Here

`outer()`

returns another function.

Therefore it is also a Higher Order Function.

Callback vs Higher Order Function

Callback Function	Higher Order Function
Passed as argument	Accepts or returns functions
Gets executed later	Controls callback execution
Does actual work	Manages other functions

Example

```
function greet(){ console.log("Hello"); }

function execute(callback){ callback(); }

execute(greet);
```

Identify
greet
↓
Callback Function
execute
↓
Higher Order Function

Easy Trick

Passed?
↓
Callback
Receives?
↓
Higher Order Function

What is Lexical Scope?

Definition

A function can access variables from where it is written, not where it is called.

Example

```
let company = "Google";

function employee(){

  console.log(company);

}

employee();
```

Output

Google

Why?

Because

employee()

was created inside the global scope.

So it can access global variables.

Another Example

```
function outer(){

  let name = "John"; function inner(){ console.log(name); } inner();

}

outer();
```

Output

John

Scope Chain

Global Scope

↓

Outer Scope

↓

Inner Scope

Inner searches upward until variable is found.

What is Closure?

Definition

A closure is created when an inner function remembers variables from its outer function even after the outer function has finished execution.

Example

```
function outer(){  
  
  let count = 0; function inner(){ count++; console.log(count); } return inner;  
  
}  
  
const counter = outer();  
  
counter();  
  
counter();  
  
counter();
```

Output

1

2

3

Question

Outer function already finished.
How is count still available?

Answer

Because of Closure.

How Closure Works Internally

1

Step 1

```
const counter = outer();
```

Memory

outer()

count = 0

↓

Returns inner()

2

Step 2

Outer finishes.

Normally

Memory Deleted

But

JavaScript notices

inner()

still needs count

So memory is NOT deleted.

3

Step 3

counter()

↓

count = 1

↓

counter()

↓

count = 2

↓

counter()

↓

count = 3

Closure keeps the variable alive.

Real World Uses of Closures

1. Counter

```
function counter(){ let value = 0; return function(){  
  value++; console.log(value); }  
}  
  
const c = counter();  
  
c();  
c();  
c();
```

Output

1
2
3

2. Private Variables

```
function bank(){  
  let balance = 5000; return function(){ console.log(balance); }  
}  
  
const user = bank();  
  
user();
```

Nobody can directly access
balance
Only the returned function can.

1. Event Listeners

```
button.addEventListener("click", function(){ console.log("Clicked");  
});
```

The callback remembers surrounding variables.
Closures are used automatically.



Interview Questions

1 Q1: What is a callback function?

Answer: A function passed as an argument to another function.

2 Q2: What is a Higher Order Function?

Answer: A function that accepts another function or returns another function.

3 Q3: Difference?

Higher Order Function

↓

Receives or returns function

Callback Function

↓

Passed into Higher Order Function

4 Q4: What is Lexical Scope?

Answer: Functions can access variables based on where they are written.

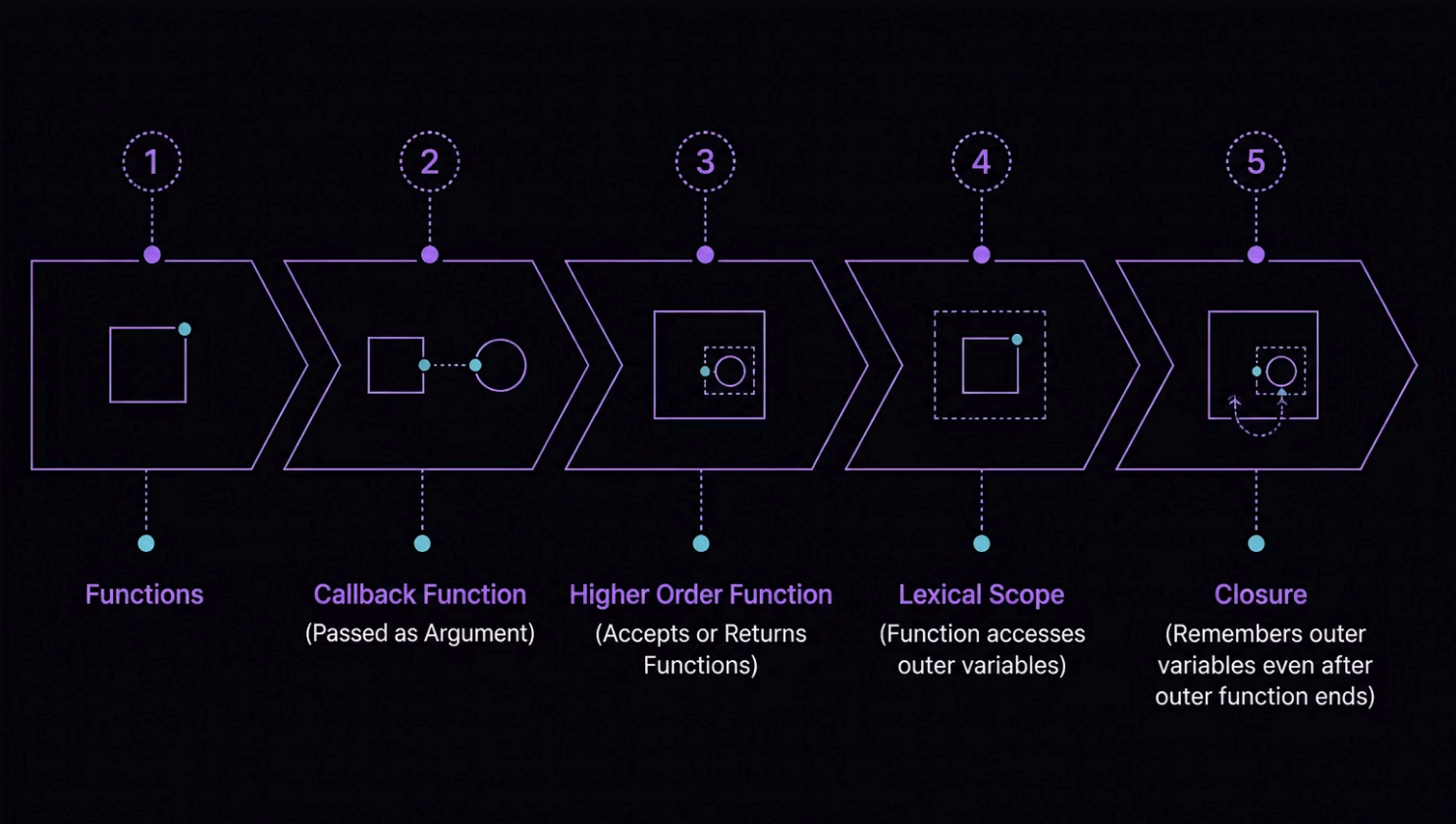
5 Q5: What is Closure?

Answer: A closure remembers variables from the outer function even after the outer function finishes execution.

6 Q6: Real-world uses?

- Data privacy
- Counters
- Event listeners
- Memoization
- Function factories
- Module pattern

Complete Mind Map



One Line Revision

Concept	Remember This
Callback	Passed into another function
Higher Order Function	Accepts or returns functions
Lexical Scope	Scope depends on where function is written
Closure	Remembers outer variables forever if needed

Final Interview Shortcut

Callback

↓
Function passed as argument

Lexical Scope

↓
Where a function is defined determines what it can access

Higher Order Function

↓
Function that accepts or returns functions

Closure

↓
An inner function remembers variables from its outer function, even after the outer function has finished execution